Interface Behavior Modeling for Automatic Verification of Industrial Automation Systems' Functional Conformance

Schwerpunktprogramm SPP 1593: Design for Future – Managed Software Evolution

Schnittstellenverhalten zur Verifikation der Funktionalen Korrektheit industrieller Automatisierungssystemen

Christoph Legat, Jakob Mund, Alarico Campetelli, Georg Hackenberg,

Jens Folmer, Daniel Schütz, Manfred Broy, Birgit Vogel-Heuser

Die Sicherstellung der Korrektheit von Modellen während der Entwicklung und Evolution von industriellen Automatisierugssystemen ist von großer Bedeutung. Eine Vielzahl unterschiedlicher Disziplinen, z.B. Mechanik, Elektrik/Elektronik und Software, sind dabei an deren Entwicklung beteiligt. In diesem Beitrag wird ein Ansatz zur Modellierung von Automatisierungssystemen vorgestellt, der die Modellierung des Schnittstellenverhaltens zur automatischen Verifikation der funktionalen Korrektheit von Artefakten adressiert und dabei Informationen von verschiedenen beteiligten Disziplinen berücksichtigt.

To facilitate engineering and evolution of automation systems, ensuring the correctness of the design models is an important topic. Industrial automation systems are composed of various heterogeneous elements designed by different disciplines such as mechanical, electrical/electronic and software engineering. In this contribution, an approach for modeling industrial automation systems is presented which is based on interface behavior modeling of design artifacts and which supports automatic verification of their functional conformance while considering information from various disciplines.

Schlagwörter: Modellbasierte Entwicklung, Schnittstellenverhalten, industrielle Automatisierungssysteme, Verifikation

Keywords: Model-based Engineering, Interface Behavior, Industrial Automation Systems, Verification

1 Introduction

Industrial plants are typically in service for more than 15 years [1]. Different disciplines, e.g. electrical/electronic, mechanical and software engineering, are involved during designing, constructing, maintaining and refining automation systems. Managing a plant's evolution has to deal with diverging evolution cycles of involved disciplines [2]. Changing system elements during evolution, e.g. adding, modifying or deleting mechanical constructions or sensors, can have discipline-specific as well as interdisciplinary influences on a the system's functionality. Managing evolution demands for ensuring correctness of the system's design. Consequently, when applying model-based (systems) engineering, managing the evolution of automation systems needs to be supported by assuring model correctness in an interdisciplinary way, i.e. considering information from various engineering disciplines. Most important, the correctness of an automation system's engineering models has to provide mechanisms for verifying the models' conformance with respect to (a set of) given requirements, i.e. ensuring requirement fulfillment. Model-based ap-

proaches as well as formal methods to verify a model's conformance to several types of requirements exist and are commonly applied in the domain of embedded systems and software. The approach presented inside this article proposes the adaption of such modeling techniques and verification methods for the specialized domain of machine and plant automation. As a first step towards realizing the model-based verification of automation systems' model correctness in the MoDEMAS project [3] the verification of functional conformance based on interface behavior is focused. In order to achieve this goal, the vision of a formal modeling framework which is based on previous works [4] and adapted for machine and plant automation is proposed that covers a requirements, a process and a system viewpoint for verifying functional conformance of automation systems that realize discrete as well continuous behavior of a production system. Therein, we focus on a system viewpoint of the automation system, which comprises the necessary aspects for verifying the functional conformance of the system to the intended functional behavior in an interdisciplinary way. Hence, a perspective on an automation system's open-loop control software enhanced with aspects of the controlled production system's mechanical and electrical/electronic behavior defining the signal chain from sensors to actuators is focused on. Thereby, aspects like error handling and operation modes of an automation system are not inside the scope of this article and will be presented in future publications. In Sect. 2, the term model correctness and related terminology such as model consistency and conformance are discussed in detail before related works on functional conformance verification are presented. A brief overview on the MoDEMAS framework is given in Sect. 3. Subsequently, in Sect. 4, an application example is introduced, which is used throughout this article to exemplify the formal modeling approach of MoDEMAS (Sect. 5). It is shown how this model is used to automatically verify functional conformance to support evolution in Sect. 6. Finally, the article is concluded in Sect. 7.

2 Model Correctness

Various terms such as model consistency, compatibility, and conformance are used in the literature to describe aspects to ensure correctness of engineering models. A brief discussion of these terms is provided in Sect. 2.1 before related work is discussed in Sect. 2.2.

2.1 Conformance, Consistency, Compatibility

Ensuring the correctness of models leads to increased success and decreased risk in development projects [5]. In literature, *consistency* is mostly defined through its opposite term *inconsistency*: Models are inconsistent if two modeling elements are contradictorily described so that they are not jointly satisfiable [6]. A model without such contradictions is called consistent [7]. The need for consistency management thus arises if models are created and maintained independently within collaborative projects [8, 9]. Within consistency management, rules define conditions evaluated to a truth value [10] and indicate whether a certain consistency is fulfilled or not. Nevertheless, ensuring full consistency among models is costly and difficult to achieve; hence, some authors state that full consistency management is impossible [11]. A field of research strongly related to consistency management is *conformance* in which models are checked against their specifications, e.g. their meta model [12]. Conformance is strongly related to requirements traceability [13], i.e. the alignment of requirements to model elements as outputs of the development process. Both functional requirements describing the intended behavior of the system and non-functional ones, e.g. dependability and usability, must be considered [14]. When engineering complex systems, the *compatibility* between two model elements must be ensured. Chakrabarti et al. [15] introduce different definitions of compatibility – both structural and behavioral ones – for checking interfaces of software modules. Interface languages are moreover used to define compatibility of components for proper composition of software modules [16] and meta models are applied for composing compatible system modules [17]. Moreover, matrix-based approaches are used for defining which elements are compatible to each other [18]. In checking the compatibility of models consisting of multiple perspectives, checking structural and behavioral compatibility is challenging [19]. Criteria such as compatibility along the lifecycle resulting from the evolution of mechatronic systems must be considered [20].

2.2 Related Works on Functional Conformance Verification

Becker et al. [21] present the *MechatronicUML* language for modeling and analyzing component-based software for mechatronic systems, which supports links between engineering disciplines. *SysML4Mechatronics* [22] is a language for interdisciplinary modeling, which addresses mechanical, electrical/electronic and software aspects explicitly. A formal semantics for automatic verification of structural compatibility has been proposed [20] but verifying functional conformance is not considered yet. Shah et al. [23] present a multi-discipline modeling framework based on SysML. The approach prescribes structural, behavioral and requirements aspects each composed into a common model. Transformations are used to map between different discipline-specific SysML profiles but verifying model correctness is not addressed. Similarly, Thramboulidis [24] proposes a viewmodel supporting interdisciplinary engineering of mechatronic systems but lacks in formal semantics for automatic verification of model correctness. An approach to automatically verify conformance of Ethernet real-time behavior by means of model-checking is proposed in

[25]. A verification platform supporting model-checking to verify conformance of PLC programs taking account of cyclic scanning mode is presented in [26]. A formal verification of PLC program correctness by means of Petri nets is presented in [27]. Further approaches to ensure behavioral correctness of logic controllers – i.e. verifying their functional conformance – with respect to given (dependability) requirements are e.g. [28, 29]. Suender et al. [30] propose an approach to ensure conformance during evolution of PLC programs by means of model-checking. An approach for automatic verification of (mid-size) instrumentation and control systems in nuclear engineering by means of model-checking is presented in [31]. An integrated approach to verify conformance of automation systems designs is presented in [32]. It considers the overall behavior of mechatronic components, i.e. the behavior resulting from the combination of different disciplines; however, an interdisciplinary model is not applied. In a nutshell, a variety of approaches to model an automation system in an interdisciplinary manner exist as well as a number of approaches to verify the correctness of the engineering models on specific aspects or disciplines (of PLCs or logic controllers). However, an interdisciplinary modeling approach that considers the information from different discipline specific models of an automation system and supports the verification of its functional behavior is still missing.

3 Overview on the MoDEMAS Framework

Within the MoDEMAS project [3], a formal modeling framework for verifying automation systems' engineering models is envisioned based on approaches from previous works [4] which are adapted for the specialized domain of machine and plant automation. MoDEMAS focuses on a formal model that contains the aspects from different engineering disciplines that can be used for verifying the model correctness. By that, evolution can be supported as subsequent to changes to the automation system's model, its correctness can be verified. The interaction between the model proposed by MoDEMAS and artifacts of typical engineering models will be investigated in later steps of the project. MoDEMAS framework is based upon three basic aspects to be modeled:

Requirements viewpoint enables to concretize from abstract aims of the automation system via design decisions to specific requirements imposed on the automation system.

Process viewpoint targets at specifying the technical process to be implemented. Therein, aspects from multiple disciplines are included describing how a specific product is manufactured. Hence, the process viewpoint describes the intended behavior the automation system shall fulfill.

System viewpoint models the actual implementation of the automation system containing the necessary aspects of involved engineering disciplines for functional conformance verification purposes. The system viewpoint is presented in detail in Sect. 5.

For modeling these viewpoints, the MoDEMAS framework relies on the FOCUS theory [4] which provides a strict formal semantics. Within FOCUS, interfaces of components play a significant role. Firstly, *static interfaces*, i.e. *typed* input and output interfaces can be used to describe compatibility between elements. Secondly, the observable behavior at a component's interface, called *semantic interface*, is defined as *stream-processing functions* which map input streams to output streams. This function is defined by (hybrid) state machines, which intents to provide suitable means for describing a hybrid system's behavior [33]. By that, formally verifying the functional conformance of the modeled interface behavior of a system with the description of the desired behavior is made possible. Since this article focuses on ensuring the functional conformance of an automation system, concepts to ensure the consistency of different views (which possibly diverge during evolution) by verifying the conformance of the interface behavior of interacting artifacts from different views will be presented in future publications.

Originally, the FOCUS theory was primarily conceived for modeling of distributed embedded systems based on a discrete time execution. Modeling automation systems holistically by considering, beside software, also e.g. mechanical aspects originate the need of a common language for the description of physical processes and phenomena. For this reason, the FOCUS theory is being extended within the MoDEMAS project towards supporting continuous time elaboration and data types [34, 35]. The behavior of hybrid components is defined by a modified version of the hybrid automaton, called I/O hybrid state machine, which was introduced in [34]. Components that have discrete as well as continuous interfaces are referred to as hybrid components. Formally, given a set of input ports I and output ports O with $I \cap O = \emptyset$ and types $T_{i \in I \cup o \in O}$, a continuous stream $\overrightarrow{c} \in \overrightarrow{I}$ is a (total) function $\overrightarrow{c} : \mathbb{R}_+ \mapsto T_i \cup \{\checkmark\}$ where $\checkmark \notin T_i$ denotes that no message occurred at the particular point in time. In contrast, discrete streams are represented as partial functions $\overrightarrow{i} : \mathbb{R}_+ \rightarrow (T_i \cup \{\checkmark\})^*$. The interface behavior is then defined as a function $F : \overrightarrow{I} \rightarrow \overrightarrow{O}$. Individual components are connected by input and output ports

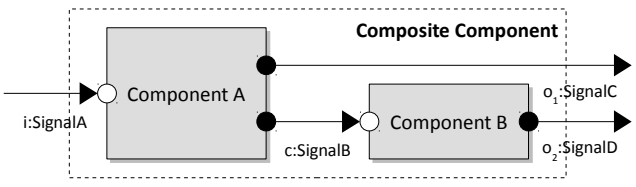

Figure 1: FOCUS in a nutshell: Two components connected via channel c forming a composite with semantic interface $F : \overrightarrow{\{i\}} \rightarrow \overrightarrow{\{o_1, o_2\}}$.

via *channels*. A well-defined *composition* operator ensures that a set of interfaces and channels form a composite component, where the interface is derived from the components and their channels enabling hierarchical specifications (cp. Fig. 1). Nevertheless, it has to be assumed that a concept for an automatic verification of the correctness of real industrial automation systems' engineering models (especially with different diverging discipline specific views) needs to abstract from the detailed engineering model's information. This article focuses on the verification of the *system viewpoint*'s functional conformance, i.e. its conformance to the process defined in the *process viewpoint*. In order to provide such an approach for automatic verification of an automation system's functional conformance, this article (i) takes into account the necessary aspects of an automation system's engineering model and (ii) integrates existing verification techniques to ensure functional conformance.

4 The Pick and Place Unit Application Example

For illustrating the concepts investigated in MoDEMAS, i.e. the formal modeling framework as well as the application of its concepts for verifying automation systems' functional conformance, the Pick and Place Unit (PPU) [36, 37], a lab-scale demonstrator which is considered suitable for evaluations of research approaches in a first step [38, 39], is used in the remainder of this article. Two different versions of the PPU are considered. In both versions, the PPU handles cylindrical workpieces (WPs) of different color (white and black). It consists of a *Stack* for storing WPs, a *Stamp*, a *Sorter*, and a *Crane* for picking and placing WPs between the others (cp. Fig. 2). In this article, it is focused on the *Sorter*.

In the PPU's *version v1*, the sorter consists of a single pneumatic cylinder and two ramps to provide two different WP withdrawal positions – one at each ramp. At first, *Ramp 1* is filled using the conveyor belt until it contains three WPs independently of the WP color. Subsequently, *Ramp 2* is filled by pushing WPs using the pneumatic cylinder into the ramp. For identifying the presence of a WP in front of the pneumatic cylinder, a light barrier is used. In *version v2*, the sorting order has to be changed to provide two WP withdrawal positions – one at each ramp – containing only WPs of one WP type per ramp. For this reason, the light barrier is replaced by an optical sensor, which identifies the WPs' types at Ramp 2. Moreover, in order to implement the new sorting order, the software behavior is adjusted accordingly.

Among others, some exemplary challenges arise when developing and evolving from PPU's version v1 to version v2. Different disciplines are tightly correlated

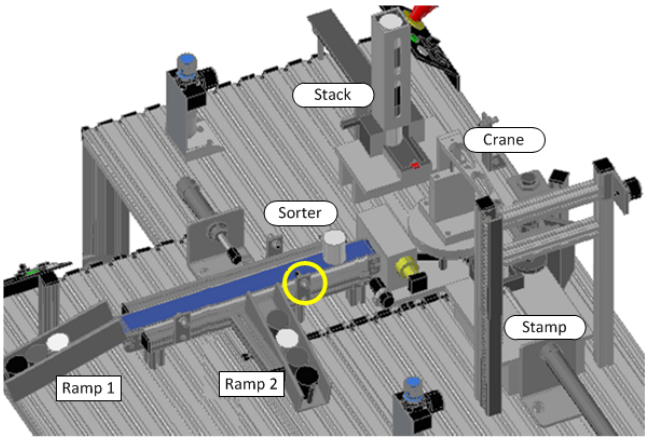

Figure 2: Overview of the applied PPU versions with highlighted sensor position at sorter (highlighted with circle)

when evolving the PPU; hence, mechanical, electrical/electronic and software aspects have to be taken into account: For example, if a respective variable (software aspect) is set, it must be ensured that the corresponding valve opens or closes (interaction of electrical/electronic and mechanic aspect) in order to extend the respective cylinder (mechanical aspects) and, by that, to fulfill the intended behavior.

The concepts of MoDEMAS are based on adapting formal modeling approaches from previous works to reflect aspects from the different disciplines to serve as a basis for verifying the functional conformance of a modeled automation system. The functional conformance can thus be ensured by taking into account aspects of the signal chain from sensor to actuator (cf. Fig. 3). In addition to the approach presented in this paper, parallel works are currently investigating concepts for ensuring that all necessary aspects to verify the functional conformance of a system's design are captured by a (new version of a) system model. When evolving from version v1 to v2, the sorter's functional conformance with respect to the adjusted sorting order (verification task VT1) should be verified in order to ensure, that the sorter behaves as intended. Additionally, to ensure that no undesired interdependencies due to the change from v1 to v2 exist, it has to be ensured that the intended behavior of unchanged elements is not affected by realized changes. For example, software commands for opening the cylinder's valve should work in both versions equivalently: the cylinder should extend (verification task VT2).

5 System Viewpoint: Interdisciplinary, Formal System Model

For enabling an automatic verification of functional conformance, the system viewpoint representing interdisciplinary aspects of the automation system is organized along the three scopes *context*, *platform* and *software*.

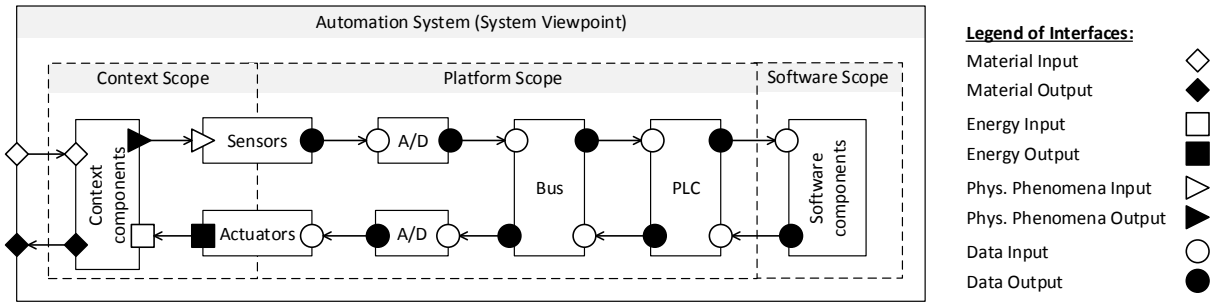

Figure 3: Schematic overview of the system viewpoint depicting an interdisciplinary control loop (visualization inspired by [40], modeling notation for interfaces based on [4])

As depicted in Fig. 3, the external observable behavior constitutes the material flow. The level of abstraction on which the automation system is modeled depends on the scope of the intended verification task: The deeper the scope of investigation, the more detailed are the formal models and the more complex are the verification tasks. The current elaboration of the approach presented here, as a first step, addresses a more abstract level of detail. More detailed and realistic as well as applicable models will be considered in future elaborations.

Context Scope provides an encapsulated workspace for describing information from the mechanical engineering discipline. The interface of the context scope therefore comprises handover positions of workpieces as well as sensor and actuator connections to the platform scope.

Platform Scope provides an encapsulated workspace for describing information from the electrical/electronic engineerÃng discipline. The model of the platform scope comprises everything from the sensor/actuator interface to the programmable controller interface with the software scope. Typically, the platform scope models (potentially unreliable) physical communication channels including effects on the data flow.

Software Scope provides a workspace for describing information from the software engineering discipline. The software scope contains a model of the software architecture including its components and their behavior. Concerning the behavior of software components, discrete state machines are used.

In the remainder of this section, details about modeling context, platform and software elements are provided.

5.1 Context Scope and its Interfaces

The *Context Scope* focuses on mechanical aspects of the interface behavior. Accordingly, continuous behavior inside modeled context elements and continuous interfaces are required. Typical elements modeled inside the context scope when designing automation systems may comprise pure mechanical components, sensors, actuators and the material flow. The behavior of context components is restricted due to physical and technical limitations. This means that in order to ensure functional conformance, the stream processing function f that represents the interface behavior of a context element obeys physical laws and is limited by the technical implementation. The objective of an automation system is to realize a desired technical process, which describes the process to produce a certain good. Therefore, modeling how a product behaves in terms of its attributes is essential to verify e.g. that an automation system realizes a desired technical process. This is referred to as material flow which is not modeled explicitly but implicitly by properties of the product, e.g. a WPs and color or position inside a context component, and aspects related to the exchange of products between context components. Hence, it is focused on the product's attributes and how these attributes are changed by the automation system. Many functional requirements can be expressed as demanded interface behavior on material ports (cf. Sect. 6). To consider the holistic design of an automation system during verification, modeling sensor and actuator behavior is inevitable. Sensors therein access information from the technical process and actuators influence the technical process [40].

In Fig. 4, a simplified excerpt of the PPU's model is shown. The *context scope* consists of pure context components (*Drive Line*, *Conveyor Belt*, *Ramps 1*, and *Ramp 2*) as well as sensors and actuators. The color of *WP*s used inside the PPU can either be black or white, i.e. color $WP.color \in B, W$ and are physically moved, i.e. they have a *position* defined by attributes that describe which context component they currently interact with and additional attributes which depend on the particular context module, e.g. $WP.pos \in \mathbb{R}$ for the conveyor belt. The interface behavior of the *Conveyor Belt* regarding the material flow can be understood as follows: material of any color enters the *Conveyor Belt* at the exact moment when placed by the crane at *WPIn* and is then transported either to output interfaces *WPOutRamp1* or *WPOutRamp2* to be handed over to *Ramp 1* (if the end of the *Conveyor Belt* is reached by the WP, i.e. $49.8 \geq WP.pos \geq 47$), or *Ramp 2* (by extending the pneumatic cylinder). The movement of the *Conveyor Belt* is realized by translatory energy of the *Drive Line* via the *BeltTransIn* port (which is sup-

posed to be either positive or zero). The *Conveyor Belt*'s behavior is given by the associated stream-processing function CB given by the hybrid state machine depicted in Fig. 4 (for the sake of simplicity, outputs on sensor S1 are not modeled).

5.2 Platform Scope and its Interfaces

The *Platform Scope* subsumes information from a variety of engineering disciplines, e.g. pneumatical, hydraulical, and electrical/electronic engineering and comprises also automation hardware. Therefore, besides discipline-specific elements, the platform scope of automation systems typically comprises sensors, actuators, field bus and physical devices that execute the control software. The elements modeled within the platform scope as well as the level of abstraction used within the models thereby depends on the scope of the verification task. In the presented approach, the interfaces of pneumatic, hydraulic and electrical/electronic components are described in terms of energy ports. Depending on the specific discipline, varying parameters might be used to model the interfaces. For example, in case of electrical/electronic components, voltage and current or aggregate energy quantities are needed while pneumatic components require e.g. air pressure. Finally, interfaces in the platform scope are modeled in terms of data input and output interfaces for enabling verification of the functional conformance. Sensors and actuators typically also comprise platform aspects. Depending on the applied field device, they might either be connected via clamps and analog/digital converters to the installed field bus system or – in case of smarter devices, which already consist of field bus interfaces – they can also be connected directly to the bus (as depicted in Fig. 4). Modeling the internal behavior, i.e. the stream processing function, is done analogously to modeling the internal behavior of the *Conveyor Belt*. The execution semantics of cyclically executed PLCs according to the IEC 61131 standard is also formulated by means of continuous timed state machines where received signals from the field bus are passed to the variables or vice versa according to the defined cycle time.

5.3 Software Scope and its Interfaces

Interfaces between *Platform* and *Software Scope* are clearly defined by the variable defined and bound to field devices within the configuration of the PLC (often referred to as hardware configuration [41]). For example, as depicted in Fig. 4, variables *var_valve* and *var_S1* are used among others for the sorting control. For modeling the internal behavior of software components, discrete state machines are used. Currently, state machines are executed step-wise in discrete time. For cyclically executed PLCs according to the IEC 61131 standard, after firing a transition, software is forced to stay within a state until the next execution cycle. In future works, the execution semantics of PLC State Charts as defined by Witsch et al. [42] will be integrated within the MoDEMAS framework. UML State Charts support the definition of further modeling notations with specialized execution semantics, e.g. by incycle states that are visited and processed within a single PLC cycle. Within this article, we solely focus on the verifying the process fulfillment, i.e. the functional conformance of the system. Aspects such as error handling and operation modes, which are important for the automation systems domain [40], will be considered in future works.

6 Supporting Evolution By Automatic Verification

In this section, it is illustrated exemplarily how the previously presented modeling approach can be used to verify the model's functional conformance, i.e. to verify that the automation system fulfills the intended behavior. The evolution of automation systems can be supported by automatic verification, if the approach is efficient, e.g. automatable through formalization, and provides sufficient means to consider engineering artifacts of multiple disciplines. Both functional and non-functional requirements need to be addressed for structural and behavioral aspects, while conformance of functional behavior has to be considered most important since it realizes the desired production process. Therefore, at first, the formalization of functional conformance issues with respect to a simplified intuitive scenario from the PPU application example is presented. Hence, contrary to approaches that rely on edit operations [43] or delta-oriented modeling [44] addressing system variants, it is shown how the proposed approach can be used to ensure functional conformance after a change and, by that, to support evolution of the automation system models. Subsequently, a brief overview on the implemented tool support is provided.

6.1 Formalizing Conformance Issues

Within this section, the formalization of conformance issues of the PPU (introduced in Sect. 4) is presented to exemplify the the proposed concepts for automatic functional conformance verification by considering simplified functional conformance aspects inside the PPU. In order to ensure the functional conformance of the PPU's version 2, the novel sorting behavior should be verified (cp. verification task VT1, Sect. 4). Therein, it is verified that the PPU does not put WPs onto a ramp of a wrong color. Therefore, a desired interface behavior for the *Conveyor Belt* component is given as:

$$CB(\overrightarrow{WPIn}, \dots) = (\overrightarrow{WPOutRamp1}, \overrightarrow{WPOutRamp2}, \dots) \Rightarrow \overrightarrow{WPOutRamp1}(t).color \neq W \wedge \overrightarrow{WPOutRamp2}(t).color \neq B$$

where CB is the conveyor belt's stream-processing function as defined in Sect. 5.1. By that, the desired behavior

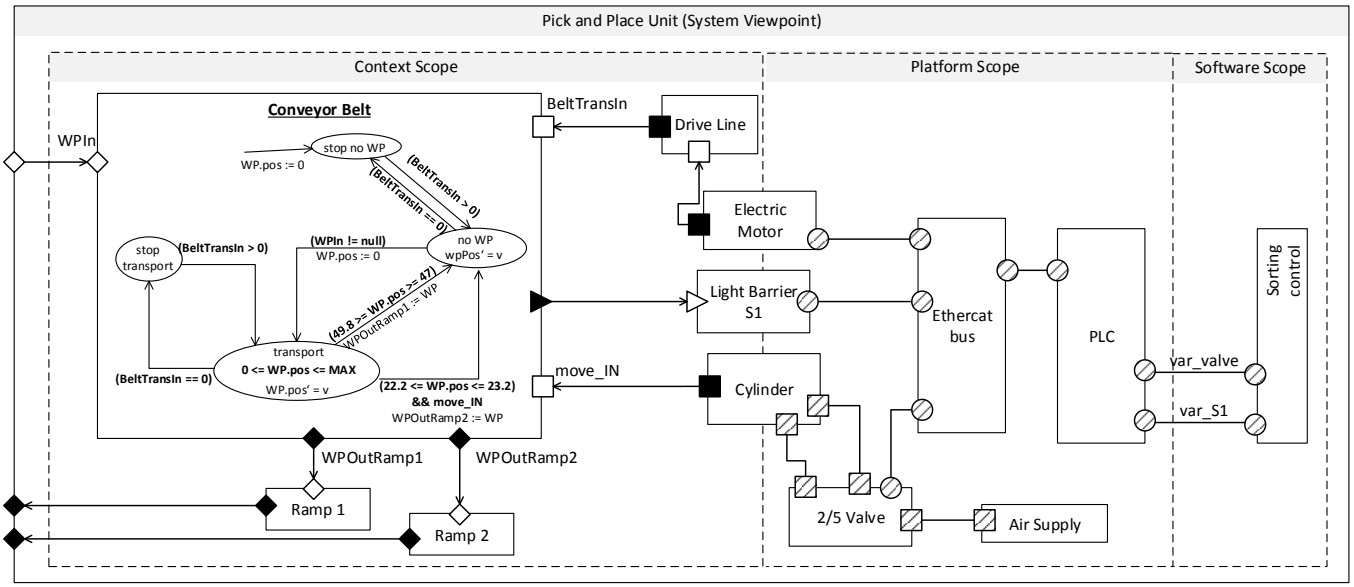

Figure 4: PPU's system viewpoint with hybrid state machine for conveyor belt behavior description (modeling notation based on [4], interfaces which are input and output are filled with diagonal lines and velocity v = sqrt(2BeltTransIn/m_{WP}) with m_{WP} the WPs' constant mass), to enhance the readability, only an excerpt of the model's information is displayed

of the material flows are verified, i.e. for any input value *WPIn* the output of the two ramps *WPOutRamp1* and *WPOutRamp2* are according to the color assumptions – black WPs (B) in Ramp 1 and white WPs (W) in Ramp 2. This interface behavior is translated to a property to be automatically verified by applying formal model-checking methods to prove its conformance with the described interface behavior of the conveyor. By that, unintentional violations of the intended sorting behavior, caused by evolutionary changes, can be detected during engineering and hence allow early adjustments.

When evolving automation systems' models, avoiding side effects of changes e.g. to system elements is important (verification task VT2, Sect. 4). In both versions of the PPU, setting the software variable *var_valve* to *true* should extend the cylinder resulting in a movement towards the conveyor (simplified as binary event *move_IN* within the example model depicted in Fig. 4), formally:

$$PL(\overrightarrow{var\_valve}, \dots) = (\overrightarrow{move\_IN}, \dots) \Rightarrow$$
$$(\overrightarrow{var\_valve}(t_0) \Rightarrow \exists t_1 > t_0 (\overrightarrow{move\_IN}(t_1)$$
$$\vee \exists t' : \ t_0 < t' < t_1 \wedge \neg \overrightarrow{var\_valve}(t')))$$

where PL is the stream-processing function for a composite component subsuming all components of the platform (including sensors and actuators). When the software variable *var_valve* is set to *true* at a respective time point t_0) and not changed meanwhile (i.e. no time point t' exists with $t_0 < t' < t_1$), a respective movement indicated by *move_IN* should be observable in a later point in time (t_1). If this property is verified, it can be ensured that no interdependencies affect the desired functionality of extending the pneumatic cylinder – the cylinder's intended behavior is ensured.

6.2 Automatic Verification in the MoDEMAS Framework

As described above, the MoDEMAS approach is based upon FOCUS I/O hybrid state machines. For verifying continuous behavior, sampling and approximation of differential equations is a well-known and established topic in communication/signal theory [45]. The time interval of the signal sampling is called period and can be constant or dynamic in the sampling execution. In general, a dynamic sampling is a more convenient solution with respect to the periodic one [46]. For FOCUS I/O hybrid state machines, two dynamic sampling algorithms to discretize their differential equations have already been developed [34]. The first approach is based on the slope of the differential function: The actual period is halved if the difference between the slope of the actual value with respect to the slope of the precedent value is high enough; the period is doubled (until a maximum value) if the slope is below a predefined value after some suitable sampling steps. A high slope can result from the actual differential equation or from a new equation associated to a different state reached in the state machine. The second approach is based on predefined critical intervals and periods derived from the system requirements. When the variable value, elaborated by the differential equation, is in an interval then a correspondent sampling period is set.

The tool realizing the MoDEMAS framework is built upon the research tool AutoFOCUS 3[1]. AutoFOCUS 3 enables a tight specification according to FOCUS and provides a user-friendly development environment for verification by integrating model checking [47]. Using the model checker NuSMV, verification as well as step-

[1] http://autofocus.in.tum.de

wise counterexample simulation is enabled. NuSMV aims at developing a state-of-the-art robust and customizable model checker, which is open source and, thus, facilitates cooperative research. The formal mapping between AutoFOCUS 3 models and NuSMV modules are explained in detail in [47]. As internal implementations of NuSMV, state machines and functional specifications are supported. Currently, AutoFOCUS 3 enables automatic verification of discrete FOCUS components. Moreover, hybrid systems are supported by HyDI – a new version of NuSMV – that enables to define continuous variables as well as derivatives and that supports the verification of invariants over continuous variables. Based on the same formal rules used for discrete components, we translated some FOCUS hybrid components into HyDI programs. Although a public version of HyDI is not available yet, the theoretical foundations for mapping between AutoFOCUS 3 and HyDI have already been laid and, hence, the mapping will be realizable in a straightforward manner.

7 Conclusion

The vision of a framework for ensuring model correctness was presented in this article. The importance of model correctness for facilitating evolution was identified and different types of model correctness were discussed at first. The modeling of interdisciplinary aspects of automation systems for automatically verifying the system's functional conformance was presented. Here, especially a model's conformity of with respect to given functional requirements was investigated. Therein, the formal modeling approach focuses on modeling the interface behavior of components based on a novel developed extension of the FOCUS theory supporting continuous interface and component behavior. Since the paper presents currently ongoing work, for a first step, some aspects are beyond the scope of this article, e.g. real-time aspects, and the electrical/electronic as well as mechanical aspects have been considered in an abstract way. By means of a simple but intuitive application example, it has been shown how this formal model can support evolution of automation systems by verifying the functional conformance of the system model. Thereby, the proposed modeling approach was applied to support evolution in two different ways: Firstly, it was used to verify that changing functional requirements are addressed correctly within the automation system's model, and secondly, for ensuring the absence of behavioral side-effects within the model. Future works will investigate the scalability of the approach and identify limitations to it by applying the approach to more complex industrial automation systems. Recently, sophisticated mechanisms for requirement and process modeling within respective viewpoints, development procedures to guide users of the approach as well as their integration in the MoDEMAS framework are being investigated. For modeling and verifying the technical process, a novel approach based on discretized observations supporting verification in early design stages is developed. To support evolving requirements, relationships between system viewpoint, requirements and engineering decisions are currently being investigated. By that, support for evolution of automation systems will be enhanced as further involved engineering disciplines as well as phases can be considered. In a next step, further mechanisms for ensuring model correctness to support evolution will be developed, e.g. compatibility and consistency within and among requirement, process and system specifications. Furthermore, techniques to improve verification performance for fine-grained, complex models need to be investigated and integrated towards our vision of a framework for ensuring model correctness.

Acknowledgments

This work was partially supported by the DFG (German Research Foundation) under the Priority Programme SPP1593: Design for Future – Managed Software Evolution

References

[1] Life-cycle-management for automation product and systems: A guideline by the system aspects working group of the ZVEI automation division, September 2012.

[2] F. Li, G. Bayrak, K. Kernschmidt, and B. Vogel-Heuser. Specification of the Requirements to Support Information Technology-Cycles in the Machine and Plant Manufacturing Industry. In *IFAC INCOM*, 2012.

[3] Research Project: Model-Driven Evolution Management Framework for Automation Systems (MoDEMAS). Funded by the German Research Foundation (Deutsche Forschungsgemeinschaft, DFG) under VO 937/20-1 within the Priority Programme SPP1593: Design for Future – Managed Software Evolution.

[4] M. Broy and K. Stølen. *Specification and development of interactive systems: Focus on streams, interfaces and refinement*. Springer, 2001.

[5] S. J. Herzig, A. Qamar, A. Reichwein, and C.J.J. Paredis. A conceptual framework for consistency management in model-based systems engineering. In *ASME International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, volume 2, pages 1329–1339, 2011.

[6] G. Spanoudakis and A. Zisman. Inconsistency management in software engineering: Survey and open research issues. *Handbook of software engineering and knowledge engineering*, 1:329–380, 2001.

[7] X. Blanc, A. Mougenot, I. Mounier, and T. Mens. Incremental detection of model inconsistencies based on model operations. In *Advanced information systems engineering*, volume 5565 of *Lecture Notes in Computer Science*, pages 32–46. Springer, 2009.

[8] E. Estevez and M. Marcos. Model-based validation of industrial control systems. *IEEE Trans. Ind. Inf.*, 8(2):302–310, 2012.

[9] L. Abele, C. Legat, S. Grimm, and A.W. Muller. Ontology-based validation of plant models. In *IEEE INDIN*, pages 236–241, 2013.

[10] P. Hehenberger, A. Egyed, and K. Zeman. Consistency checking of mechatronic design models. In *ASME International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, volume 3, pages 1141–1148, 2010.

[11] A. Qamar, C. J.J. Paredis, J. Wikander, and C. During. Dependency modeling and model management in mechatronic design. *Journal of Computing and Information Science in Engineering*, 12(4), 2012.

[12] R. F. Paige, P. J. Brooke, and J. S. Ostroff. Metamodel-based model conformance and multiview consistency checking. *ACM Trans. on Software Engineering and Methodology (TOSEM)*, 16(3), 2007.

[13] T. N. Nguyen and E. V. Munson. A model for conformance analysis of software documents. In *International Workshop on Principles of Software Evolution*, pages 24–35. IEEE, 2003.

[14] M. A. Wehrmeister, C. E. Pereira, and F. J. Rammig. Aspect-oriented model-driven engineering for embedded systems applied to automation systems. *IEEE Trans. Ind. Inf.*, 9(4):2373–2386, 2013.

[15] A. Chakrabarti, L. De Alfaro, T. A. Henzinger, M. Jurdziński, and F. Mang. Interface compatibility checking for software modules. In *Computer Aided Verification*, volume 2404 of *Lecture Notes in Computer Science*, pages 428–441. Springer, 2002.

[16] D.C Craig and W.M. Zuberek. Compatibility of software components-modeling and verification. In *International Conference on Dependability of Computer Systems*, pages 11–18, 2006.

[17] Bergen Helms and Kristina Shea. Computational Synthesis of Product Architectures Based on Object-Oriented Graph Grammars. *Journal of Mechanical Design*, 134(2), 2012.

[18] C.R. Bryant, R.B. Stone, D.A. McAdams, T. Kurtoglu, and M.I. Campbell. Concept generation from the functional basis of design. In *Int. Conf. on Engineering Design*, 2005.

[19] S. Feldmann, C. Legat, K. Kernschmidt, and B. Vogel-Heuser. Compatibility and coalition formation: Towards the vision of an automatic synthesis of manufacturing system designs. In *IEEE Int. Symp. on Industrial Electronics*, 2014.

[20] S. Feldmann, K. Kernschmidt, and B. Vogel-Heuser. Combining a SysML-based modeling approach and semantic technologies for analyzing change influences in manufacturing plant models. In *CIRP CMS*, 2014.

[21] S. Becker, C. Brenner, S. Dziwok, T. Gewering, C. Heinzemann, U. Pohlmann, C. Priesterjahn, W. Schäfer, J. Suck, O. Sudmann, and M. Tichy. The mechatronicuml method – process, syntax, and semantics. Technical Report TR-RE-12-318, Software Engineering Group, Heinz Nixdorf Institute University of Paderborn, 2012.

[22] K. Kernschmidt and B. Vogel-Heuser. An interdisciplinary sysml based modeling approach for analyzing change influences in production plants to support the engineering. In *9th IEEE CASE*, 2013.

[23] A. A. Shah, A.A. Kerzhner, D. Schaefer, and C.J.J. Paredis. Graph transformations and model-driven engineering. In *Multi-view modeling to support embedded systems engineering in SysML*, pages 580–601, 2010.

[24] K. Thramboulidis. The 3+1 sysml view-model in model integrated mechatronics. *IEEE Transactions on Industrial Informatics*, pages 109–118, 2010.

[25] D. Witsch, B. Vogel-Heuser, J.-M. Faure, and G. Marsal. Performance analysis of industrial ethernet networks by means of timed model-checking. In *IFAC INCOM*, 2006.

[26] S. Biallas, J. Brauer, and S. Kowalewski. Arcade.PLC: A verification platform for programmable logic controllers. In *IEEE/ACM Int. Conf. on Automated Software Engineering*, pages 338–341, 2012.

[27] T. Mertke and G. Frey. Formal verification of PLC programs generated from signal interpreted petri nets. In *IEEE Int. Conf. on Systems, Man, and Cybernetics*, volume 4, pages 2700–2705, 2001.

[28] J.J.B. Machado, B. Denis, J.-J. Lesage, J.-M. Faure, and J. Fereira. Logic controllers dependability verification using a plant model. In *IFAC Workshop on Discrete-Event System Design*, 2006.

[29] V. Gourcuff, O. De Smet, and J.-M. Faure. Improving large-sized PLC programs verification using abstractions. In *IFAC World Congress*, 2008.

[30] C. Sünder, V. Vyatkin, and A. Zoitl. Formal verification of downtimeless system evolution in embedded automation controllers. *ACM Transactions on Embedded Computing Systems (TECS)*, 12(1):17, 2013.

[31] J. Lahtinen, J. Valkonen, K. Björkman, J. Frits, I. Niemelä, and K. Heljanko. Model checking of safety-critical software in the nuclear engineering domain. *Reliability Engineering & System Safety*, 105:104–113, 2012.

[32] V. Vyatkin, H.-M. Hanisch, C. Pang, and C. Yang. Closed-loop modeling in future automation system engineering and validation. *IEEE Trans. on Systems, Man, and Cybernetics, Part C*, 39(1):17–28, 2009.

[33] B. De Schutter, W.P.M.H. Heemels, J. Lunze, and C. Prieur. *Survey of modeling, analysis and control of hybrid systems*, pages 31–55. Cambridge University Press, 2009.

[34] A. Campetelli. Dynamic Sampling for FOCUS Hybrid Components. *International Journal of Modeling and Optimization*, 3(5):402–406, 2013.

[35] M. Broy. System behaviour models with discrete and dense time. In *Advances in Real-Time Systems*, pages 3–25. Springer, 2012.

[36] B. Vogel-Heuser, C. Legat, J. Folmer, and S. Feldmann. Researching evolution in industrial plant automation: Scenarios and documentation of the pick and place unit. Technical Report TUM-AIS-TR-01-14-02, Institute of Automation and Information Systems, Technische Universität München, 2014.

[37] Institute of Automation and Information Systems, Technische Universität München. The pick and place unit demonstrator for evolution in industrial plant automation. Online available: http://www.ppu-demonstrator.org, 2014.

[38] Birgit Vogel-Heuser, Jens Folmer, and Christoph Legat. Anforderungen an die Softwareevolution in der Automatisierung des Maschinen- und Anlagenbaus. *at – Automatisierungstechnik*, –(3):163–174, 2014.

[39] Birgit Vogel-Heuser, Christoph Legat, Jens Folmer, and Susanne. Rösch. Challenges of Parallel Evolution in Production Automation Focusing on Requirements Specification and Fault Handling. *at – Automatisierungstechnik – Special Issue on SPP1593*, –:–, 2014.

[40] B. Vogel-Heuser, C. Diedrich, A. Fay, S. Jeschke, S. Kowalewski, M. Wollschlaeger, and P. Göhner. Challenges for software engineering in automation. *Journal of Software Engineering and Applications*, 7(5), 2014.

[41] E. Estévez, M. Marcos, and D. Orive. Automatic generation of PLC automation projects from component-based models. *Journal of Advanced Manufacturing Technology*, 35(5-6):527–540, 2007.

[42] D. Witsch and B. Vogel-Heuser. PLC-statecharts: An approach to integrate UML-statecharts in open-loop control engineering – aspects on behavioral semantics and model-checking. In *IFAC World Congress*, 2011.

[43] T. Kehrer, U. Kelter, M. Ohrndorf, and T. Sollbach. Understanding model evolution through semantically lifting model differences with SiLift. In *IEEE Int. Conf. on Software Maintenance*, 2012.

[44] M. Kowal, I. Schaefer, and M. Tribastone. Family-Based Performance Analysis of Variant-Rich Software Systems. In *Fundamental Approaches to Software Engineering*, volume 8411 of *Lecture Notes in Computer Science*. Springer, 2014.

[45] J. Stoer, R. Bulirsch, R H. Bartels, W. Gautschi, and C. Witzgall. *Introduction to numerical analysis*. Texts in applied mathematics. Springer, 2002.

[46] S. Ilie, G. Söderlind, and R. M. Corless. Adaptivity and computational complexity in the numerical solution of ODEs. *J. Complexity*, 24(3):341–361, 2008.

[47] A. Campetelli, F. Hölzl, and P. Neubeck. User-friendly Model Checking Integration in Model-based Development. In *24th International Conference on Computer Applications in Industry and Engineering*, 2011.

Manuscript Delivery: 15. September 2014.

Dipl.-Inf. Christoph Legat is a researcher at the Institute of Automation and Information Systems, Technische Universität München. His research interest is on the application of formal methods and knowledge-based techniques to improve the flexibility and changeability of automation control systems.

Adresse: Technische Universität München, Institute of Automation and Information Systems, Boltzmannstr. 15, 85748 Garching bei München. E-mail: legat@ais.mw.tum.de

M.Sc. Jakob Mund works as a research assistant at the Chair IV: Software & Systems Engineering at Technische Universität Müenchen. His research interests include model-driven engineering and the application of formal methods.

Adresse: Technische Universität München, Institut für Informatik - Chair IV, Boltzmannstr. 3, 85748 Garching bei München. E-mail: mund@in.tum.de

M.Sc. Alarico Campetelli is a researcher at Chair IV: Software & Systems Engineering at Technische Universität Müenchen. His research interests are in formal methods, formal verification, hybrid systems and model-based development.

Adresse: Technische Universität München, Institut für Informatik - Chair IV, Boltzmannstr. 3, 85748 Garching bei München. E-mail: campetel@in.tum.de

M.Sc. Georg Hackenberg is a researcher at Chair IV: Software & Systems Engineering at Technische Universität Müenchen. His research interest is on model-based approaches for the development of cyber-physical systems with special focus on mechatronics and dynamic optimization.

Adresse: Technische Universität München, Institute für Informatik - Chair IV, Boltzmannstr. 3, 85748 Garching bei München. E-mail: hackenbe@in.tum.de

M. Sc. Jens Folmer is currently researching towards his Ph.D. degree at the Institute of Automation and Information Systems, Technische Universität München. He does research on the formal analysis of process and alarm data for cause-effect analysis and root cause detection and diagnosis in automation.

Adresse: Technische Universität München, Institute of Automation and Information Systems, Boltzmannstr. 15, 85748 Garching bei München. E-mail: folmer@ais.mw.tum.de

Dipl.-Ing. Daniel Schütz is currently researching towards his PhD degree at the Institute of Automation and Information Systems, Technische Universität München. He does research on the design of model-based development of intelligent real-time software for automation systems with regard to plant dependability and modularity aspects.

Adresse: Technische Universität München, Institute of Automation and Information Systems, Boltzmannstr. 15, 85748 Garching bei München. E-mail: schuetz@ais.mw.tum.de

Prof. Dr. Dr. h.c. Manfred Broy studied Mathematics and Computer Science at the Technische Universität München. In 1980 he received his Ph.D. and 1982 he completed his Habilitation Thesis. Since 1989 he is a full professor for computer science at the Faculty of Computer Science at the Technische Universität München, Chair IV: Software & Systems Engineering (former chair of Professor F.L. Bauer). His research interests are software and systems engineering comprising both theoretical and applied aspects including system models, specification and refinement of system components, specification techniques, development methods and verification. In 1994 he received the Leibniz Award by the Deutsche Forschungsgemeinschaft and in 2007 the Konrad Zuse Medal by the Gesellschaft für Informatik.

Adresse: Technische Universität München, Institute für Informatik - Chair IV, Boltzmannstr. 3, 85748 Garching bei München. E-mail: broy@in.tum.de

Prof. Dr.-Ing. Birgit Vogel-Heuser graduated in electrical engineering and received the Ph.D. in mechanical engineering from the RWTH Aachen in 1991. She worked for nearly ten years in industrial automation in the machine and plant manufacturing industry. After holding different chairs of automation she has been head of the Institute of Automation and Information Systems at the Technische Universität München since 2009. Her research work is focused on modeling and education in automation engineering for distributed and intelligent systems.

Adresse: Technische Universität München, Institute of Automation and Information Systems, Boltzmannstr. 15, 85748 Garching bei München. E-mail: vogel-heuser@ais.mw.tum.de